Evidence-First Design of a Distributed Fraud Detection Engine

Shared-Memory ABI, Deadline-Aware Batching, and GPU Execution Methodology

Project research note
Implementation-aligned design document | 30 July 2026

Abstract

Fraud scoring on an authorization path is judged by tail latency, bounded failure behavior, and auditability as much as model accuracy. This paper documents an implementation-aligned design: a Rust gateway serializes a fixed request into a System V shared-memory slot; a native daemon consumes it through a bounded MPMC ring; and a deadline-aware batcher dispatches compact batches to a capability- gated GPU path. The design separates the synchronous path from streaming aggregation, training, and rollout. It specifies a stable ABI, allocation boundaries, sequence-number ownership, and a reproducible protocol based on open-loop Poisson arrivals and HDR histograms. Two bounded evidence sets are reported. Five local RTX 3050 component runs found no benefit for the tiny launch-dominated graph stub: direct and graph-replay p99 were 58.1 and 59.5 μs. One seeded Linux CPU-reference run completed 15,000 authenticated HTTP/shared-memory decisions at 1,500 requests/s with no errors, p50 2.325 ms, p99 3.421 ms, and p99.9 3.939 ms. These are not full-model production results; V0=150 ms through V3=4 ms remain historical/design targets.

Table 1: Latency scope and current evidence status.

Scope	Definition and status
Kernel	Nsight GPU activity for the warmed forward stub: median 0.864 μs ($n = 2{,}100$); not a full model.
Batch call	CPU wall time around warmed batch-32 copy/forward/synchronize: direct p99 58.1 μs; graph p99 59.5 μs.
API decision	Planned client arrival through HTTP response on one CPU-only GitHub runner: p99 3.421 ms ($n = 15{,}000$).

Table 2: Historical target ledger, not measured results.

Version	Value	Stage	Interpretation
V0	150 ms	naive PyTorch API	historical target
V1	45 ms	C++ API / TCP	historical target
V2	12 ms	shared-memory IPC	historical target
V3	4 ms	graph / pinned memory	historical target
V4	–	calibrated low precision	definition needed
V5	–	fused short context	definition needed

1 Problem and service-level boundary

An authorization decision must arrive before its caller's deadline, with clear provenance for both primary and degraded outcomes. The engine returns an allow, review, or decline recommendation from a bounded transaction representation. Its low-latency objective applies only to a warm, co-located scoring path inside one region. Ingestion, feature backfill, training, promotion, cold start, and cross-region calls are outside that boundary. Tail latency is whole-path behavior: a fast kernel cannot compensate for queueing, cache misses, or a network hop.

The target inherits the tail-at-scale problem: fan-out makes an infrequent slow component visible to a user-facing tail [1]. Each request has one absolute deadline. Gateway validation, one fixed-width feature read, scheduler queueing, worker dispatch, and encoding consume shrinking child budgets; work is not begun after expiration. A timeout or unhealthy worker returns a deterministic, auditable fallback when a trustworthy feature block exists, and fails closed when it does not.

1.1 Historical versions are not results

The project brief's V0–V5 sequence is a decision plan. V0 (naive PyTorch API), V1 (C++ API over TCP), V2 (shared-memory IPC), and V3 (CUDA Graphs plus pinned memory) identify intended comparisons; V4 and V5 need frozen definitions before measurement. The values V0=150 ms, V1=45 ms, V2=12 ms, and V3=4 ms are historical/design targets only. No like-for-like manifest proves them, so they are neither observed latency nor an SLA. The repository's 1 ms warm/co-located phase budget is also prospective.

2 Architecture

The warm path authenticates and bounds the payload, materializes one fixed-width feature vector, reserves a daemon slot, and waits only until the inherited deadline. The daemon batches ready slots and executes a CPU reference handler or a separately enabled CUDA/CUTLASS integration. The stream processor is off the critical path: it maintains event-time state and writes materialized features for later reads.

The deployment model uses ready replicas, health/circuit routing, immutable model artifacts, TLS-protected depen-

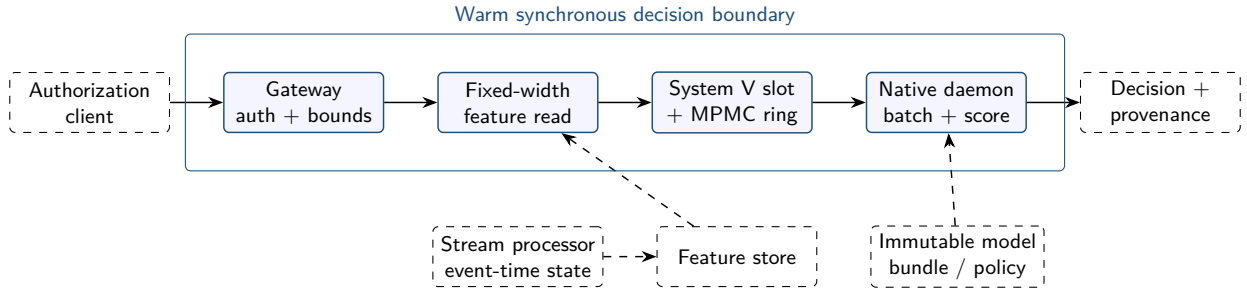

Figure 1: Architecture and latency boundary. Dashed elements affect correctness or availability but are not sequential warm-request stages.

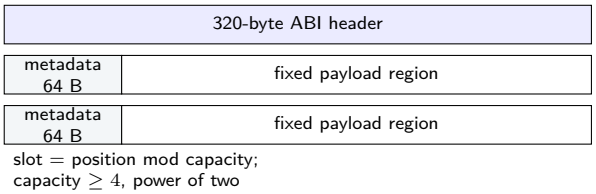

Figure 2: Shared segment layout. Serialization targets the selected slot's fixed backing region.

dencies, and explicit fallback labels. It does not promise exactly-once multi-writer aggregation; Kafka transactions, durable state, and Redis version fencing remain prerequisites. Nor does it claim the local CPU scorer is an exported production transformer.

3 ABI and zero-copy handoff

The interprocess contract is a versioned binary ABI, not a C++ object layout. The daemon creates a mode-0600 System V segment; the gateway locates it with `shmget` and attaches with `shmat`. Their lifecycle and permission semantics are documented by Linux [6, 7]. The header begins with magic `0x46444950` (FDIP), version 1, a 320-byte header, and a power-of-two slot count. Atomic fields are explicitly aligned integer storage, avoiding an unstable cross-language `std::atomic` representation.

The gateway's FlatBuffers builder uses a fixed, non-growing allocator over that payload. Allocation either fits or the request is rejected before queue reservation. Zero-copy is limited to the gateway–daemon handoff: no intermediate serialized `Vec` is created, but parsing and other system copies remain. FlatBuffers documents custom allocation in its Rust builder API [5]. Fixed layout, bounds, and version validation remain necessary; shared memory is not a trust boundary. The daemon bounds-checks the slot-relative range, verifies the FRTX buffer with the generated C++ binding, checks required strings and request-ID agreement, and then borrows views into the slot without materializing a request object.

4 No-allocation daemon and SLA batcher

The scheduler hot path owns no growing containers. It uses a fixed-capacity array of slot pointers, a preallocated pool for eligible temporary memory, and a `noexcept` handler. The native tests replace global allocation operators and assert no allocation during one `poll_once()` call. This is a narrow implementation assertion, not a claim that metrics, drivers, logging, or every production scoring component never allocates. Graph replay is separately designed to avoid per-replay stream creation and host allocation after startup.

The continuous batcher drains ready slots to a maximum of 32 in the daemon executable. It calls the handler when full or when the oldest element has waited at least its configured 2 ms deadline. For oldest enqueue time e_0, monotonic time t, capacity B, and staged count n, the trigger is

$$(n = B) \vee (t - e_0 \geq D), \qquad 0 < n \leq B.$$

This bounds scheduler waiting only if `poll_once()` is called frequently. It does not bound store work, GPU work, OS scheduling, or end-to-end response latency. Full-batch and deadline-batch counters let experiments relate throughput to the trigger that dispatched each batch.

5 Lock-free ring correctness

The ring is in the per-cell sequence-number family of bounded MPMC queues [11, 12]. A producer observes enqueue position p, claims it by compare-and-exchange only when the slot sequence equals p, writes its payload and metadata, then release-publishes $p + 1$. A consumer claims p after acquire-observing $p + 1$. Completion writes the response and release-publishes $p+2$. The original producer owns that response until it copies it and stores $p+C$, releasing the slot for the next ring generation, where C is capacity.

The argument is deliberately narrow. Unique CAS ownership prevents duplicate claims. Release/acquire synchronization exposes preceding writes to the acquirer. The completed state prevents early response overwrite. Power-of-two capacity and absolute positions distinguish generations in the intended range. Tests exercise concurrent production, consumption, and wraparound, but are neither a

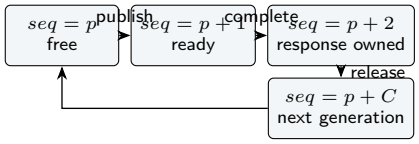

Figure 3: Per-slot state progression for absolute position p. A response cannot be overwritten before its producer releases it.

formal proof nor a cross-architecture memory-model proof. A crashed producer may leave a hole; abandoned-slot recovery needs a separate protocol. Lock freedom here does not mean the feature store, GPU driver, handler, or network cannot block.

6 CUDA Graph and CUTLASS methodology

CUDA Graphs can reduce repeated CPU launch overhead by capturing a stable operation sequence and replaying an instantiated executable graph. This design pre-captures batch sizes 1 through 32 with persistent non-blocking streams and host/device buffers. The captured reference sequence is H2D copy, forward stub, and D2H copy. CUDA defines graph capture and asynchronous execution semantics that a benchmark must honor [2, 3].

This is a methodology hook, not evidence that the exported model uses graph replay. A valid experiment compares identical shapes and buffers in direct and replay modes after warmup, separately reports capture cost and uncaptured-shape misses, and uses CUDA events for kernel/GPU-interval timing. Nsight Systems and Compute establish launch behavior, occupancy, memory traffic, and stall reasons [16].

CUTLASS is optional rather than silently substituted at runtime. The current wrapper configures an SM80 TensorOp INT8-to-INT32 GEMM with 128×128×64 threadblock, 64×64×64 warp, and 16×8×32 instruction shapes. CUTLASS supplies composable CUDA GEMM kernels [4]; its presence does not establish model numerical equivalence, cuBLASLt comparison, or Tensor Core throughput on a named device. FP8 candidates must measure score drift and decision flips against FP32. FP8 formats and scaling trade-offs are described by Micikevicius et al. [14]. The current E4M3 code uses FP8 storage/conversion with FP32 accumulation and makes no Tensor Core throughput claim.

7 Load methodology: Poisson and HDR

The load generator uses seeded exponential inter-arrival times: an open-loop Poisson process with mean rate λ. Latency begins at the *planned* arrival, not worker admission, retaining saturation-induced client queueing. Concurrency bounds in-flight sockets/goroutines without replacing the arrival process. Entity mix, rate, duration or

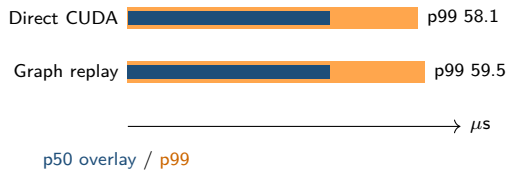

Figure 4: Comparable local CUDA component percentiles generated from checked-in manifests. The generated table also reports the separately scoped Linux API run; targets are never interpolated into either view.

request count, and seed are run inputs.

HDR Histogram records every completion, including transport failures and non-2xx responses, with status counts kept separately. Because observations start at planned arrivals, the tester uses `RecordValue`, not synthetic coordinated- omission correction. HDR Histogram is designed for wide ranges at controlled precision [8]. Report offered load, achieved throughput, error/fallback rate, warmup, duration, and per-phase delay, not only a favorable percentile. Orca provides context for iteration-level scheduling and selective batching in latency-sensitive inference serving [9]. Triton's dynamic batching documentation is a comparison point for batching policies, not a claim of Triton equivalence [15].

8 Results: checked-in evidence only

3 completed measured configuration(s) were generated from checked-in manifests; scope labels below are binding.

At commit `2627824c`, five independently launched component runs each used 100 warmups and 2,000 observations on an RTX 3050. Median run summaries were p50/p99 40.4/58.1 μs for direct execution and 40.5/59.5 μs for graph replay. Thus CUDA Graph replay did not improve p50 or p99 for this deliberately small stub. Compute Sanitizer reported zero errors, and the single CUTLASS INT8–INT32 GEMM matched its CPU reference exactly; neither establishes model accuracy or sustained Tensor Core throughput.

At commit `d71b52fd`, a GitHub-hosted four-vCPU AMD EPYC runner drove one seeded open-loop Poisson run through Rust HTTP parsing, direct FlatBuffer serialization, System V IPC, C++ FRTX verification/decoding, the scheduler, and the CPU reference handler. All 15,000 requests returned HTTP 200. Achieved throughput was 1,505.70/s; p50, p90, p99, p99.9, and maximum planned-arrival-to- completion latency were 2.325, 3.145, 3.421, 3.939, and 4.943 ms. This is an observational integration run, not a capacity limit or repeated comparison.

Nsight Systems observed positive host/GPU overlap in 500 of 500 bounded probes; median overlap was 362 ns. Nsight Systems also found asynchronous H2D API return before device-copy completion in 2,093 of 2,100 joined calls. Nsight Compute hardware counters were unavailable under `ERR_NVGPUCTRPERM`; occupancy, Tensor Core

Table 3: Minimum reproducibility record. A benchmark is incomplete if required fields are absent.

Category	Required fields	Category	Required fields
Identity	commit, UTC time, raw artifact paths	Hardware	CPU, GPU/count, network
Software	OS, driver, CUDA, compiler	Model	immutable hash, shape, backend
Load	duration, warmup, rate, seed, arrival, queue cap	Outcomes	percentiles, throughput, errors, fallbacks
Comparison	equal corpus/shapes, five repeats	Profiling	CUDA events, Nsight, cold/warm split

Table 4: Measured summaries generated from checked-in JSON manifests.

Artifact	Commit	Backend	Scope	p50 (μs)	p99 (μs)	Rate/s	Errors
CUDA direct runs	2627824c	Direct CUDA	CPU wall time per warmed direct batch-32 H2D/forward/D2...	40.4	58.1	–	0.00%
CUDA graph runs	2627824c	Graph replay	CPU wall time per warmed batch-32 replay, including cud...	40.5	59.5	–	0.00%
Linux HTTP/IPC run	d71b52fd	HTTP + IPC	planned open-loop client arrival through authenticated ...	2325	3421	1505.70	0.00%

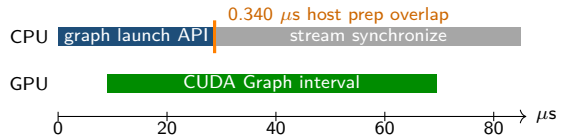

Figure 5: Measured Nsight Systems sample on the RTX 3050. The host prepared bounded response metadata after `cudaGraphLaunch` returned and before synchronization; the 0.340 μs interval lies inside the device graph interval. This is real but too small to imply an end-to-end speedup.

utilization, cache traffic, and stall metrics are therefore not claimed.

The Linux workflow also retained 4,988 daemon-only `perf` samples, folded stacks, and a rendered flame graph. Sampled time is dominated by the polling loop, monotonic clock reads, and scheduler yielding; no allocation symbol appears in the folded sample. This supports—but cannot prove—the hot-path allocation claim, because sampling can miss short calls and excludes the gateway and client. The allocation-counting unit test is the direct regression guard. The SVG, folded stacks, profiler diagnostics, and provenance are checked in under `docs/profiling/results/`.

Performance promotion is a gated experiment: at least five repeated runs should show numerical tolerance, no API p99 regression at equal offered load, bounded deadline-miss/fallback rate, and tested capability fallback. MLPerf Inference is a useful reference for controlled scenarios, accuracy constraints, and reporting discipline [10]; this project does not claim MLPerf compliance.

9 Threats, limitations, and reproducibility

Model validity. Deterministic/synthetic lifecycle material does not establish production fraud quality, fairness, calibration under drift, or economic cost. TabTransformer motivates contextual tabular modeling but does not validate this feature set or model [13].

Systems validity. The local worker is a deterministic scorer, not the exported transformer. The graph path captures a forward stub. System V IPC is Linux-specific, and CPU/GPU timing is not API timing. Warm co-located results would not generalize to remote clients, cache misses, contention, driver changes, or failure recovery.

Concurrency validity. Lock freedom applies to the bounded ring's progress condition, not to the whole deployment. The batch handler, feature store, GPU driver, memory allocator outside the guarded scheduler, and network may block. Tests are valuable regressions, but no formal linearizability proof or cross-process model checking is provided.

Security and operations. Mode-0600 constrains segment access but does not authenticate a compromised local process. Production still needs host hardening, secret rotation, signed models, access control, safe telemetry, and reviewable fallback policy. Fail-closed behavior trades availability for integrity when trustworthy features are absent.

To reproduce an eventual claim: (1) build a named commit and record hardware, software, capability selection, and model hash; (2) warm keys and graph variants, while retaining cold runs separately; (3) drive seeded Poisson load and collect HDR histograms, quantile CSVs, status counts, CUDA events, and Nsight exports; (4) place completed JSON/CSV artifacts under `docs/profiling/results/`; and (5) run `python scripts/generate_research_figures.py` before compiling this paper. Compare baseline/candidate at equal offered load and corpus, retain failed runs, and impair a worker/store. The checked-in artifacts now satisfy this protocol for the two narrow scopes reported above. Inputs still needed before a production conclusion are a real model bundle and corpus, repeated paired API baselines, cold-start and failure runs, numerical decision-drift analysis, and permitted Nsight Compute counters.

Table 5: Required comparisons before a latency or throughput conclusion.

Experiment	Controls and candidate	Required observations
CPU reference	fixed corpus, shape, load; FP32 handler	API percentiles, errors, fallback, CPU saturation
CUDA direct	same warmed buffers; direct H2D/forward/D2H	kernel and GPU interval, drift, capability
Graph replay	same CUDA inputs; captured batch 1–32	replay, capture cost, misses, API p99
CUTLASS GEMM	same matrices/tolerance; optional INT8	correctness, throughput, occupancy, traffic
Batching	same offered rate; batcher versus one	queue delay, trigger counts, p99, throughput
Failure path	same workload; worker/store impairment	misses, fallbacks, recovery, provenance

10 Conclusion

This design makes ownership explicit at the shared-memory boundary, keeps the daemon scheduler bounded and allocation-aware, uses an SLA batch trigger, and treats CUDA Graph/CUTLASS integrations as experiments rather than performance slogans. Its most useful current component result is negative: graph replay did not improve the p50 or p99 of the tiny launch-dominated stub. The CPU-only Linux run demonstrates the complete authenticated HTTP-to-shared-memory protocol at the stated load, while deliberately stopping short of a production-model or capacity claim. Keeping targets separate from measured scope leaves a clear route to credible evaluation.

11 Evaluation matrix for future runs

The purpose of an evaluation matrix is to prevent a system change from being credited for a difference caused by another changed variable. Every row below is a required comparison, not an existing measurement. The baseline and candidate must use the same request corpus, model bundle, feature shape, offered load, warmup rule, and error policy. The output is written to the machine- readable manifest only after raw artifacts are preserved.

An API conclusion must remain distinct from a component conclusion. CUDA events can demonstrate a kernel or GPU interval only; they cannot measure request parsing, feature retrieval, socket queueing, scheduler delay, response encoding, or the client network path. Conversely, an API percentile without phase breakdown cannot establish why it changed. The protocol therefore records both types of evidence and labels their scope at report time.

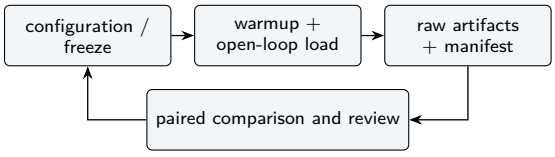

Figure 6: Evidence loop for each optimization. A result is reviewable only after configuration, raw artifacts, and paired comparison agree.

Table 6: Decision-integrity evidence to retain beside timing artifacts.

Item	Evidence	Rejection condition
Reference inputs	immutable corpus ID, seed, fixed feature shape	corpus differs across compared paths
Calibration	bundle hash, thresholds, precision/scales, declared tolerances	undeclared recalibration after observing output
Scores	max absolute/relative error, distribution summary, non-finite count	tolerance breach or unexplained non-finite
Decisions	allow/review/decline confusion matrix and changed request IDs	unreviewed decision flip
Fallback	device capability, reason, backend reported to caller	fallback counted as accelerated primary result
Repeatability	five paired runs and artifact hashes	a favorable single run is presented as conclusive

12 Numerical and decision-integrity protocol

Latency optimization is not useful if it changes decisions silently. The FP32 reference serves as a component-level oracle for finite inputs. Any FP16, FP8, or INT8 candidate must declare its calibration/tolerance policy before the run, then report absolute and relative score error, changed decision count, changed explanation count where applicable, and all non-finite outputs. A capability failure must recompute or report the documented fallback; it must not be labeled as a successful accelerated result merely because a device advertises support.

This policy is especially important for FP8. The project may use E4M3 storage with FP32 accumulation on capable hardware, but that implementation fact does not certify accuracy or Tensor Core performance. The FP8 literature establishes formats and scaling techniques, not this deployment's calibration [14]. The same separation applies to tabular-model literature: TabTransformer is motivation for architecture choices, not evidence of fraud quality here [13].

Table 7: Controlled impairment plan for the warm decision path.

Impairment	Injection and expected behavior	Record
Worker removal	router opens circuit; reroute or fallback	route, reason, remaining deadline
Feature-store delay	child deadline; fail closed without features	phase delay, status, decision
Ring saturation	bounded reject/queue deadline; no corrupt slot	depth, sequence, error count
CUDA capability loss	explicit CPU/fallback; no false acceleration	capability, backend, score drift
Graph miss	direct path or explicit unavailable state	graph status, batch size, timing scope
Client overload	planned-arrival latency exposes queueing	offered/achieved rate, HDR percentile

13 Failure and tail-latency runbook

Tail behavior must be measured under steady load and under controlled loss. The following injections are bounded and reversible in a test environment. They should never be executed against a live authorization service without an approved safety plan. The expected behavior is an observable deadline/fallback transition, not a silent extension of the caller deadline.

Tail aggregation must report each population separately: warm primary success, cold or uncaptured startup, explicit fallback, feature failure, timeout, and transport error. Combining them can conceal a failure mode or make a healthy subpopulation look worse than its contract. The design follows the tail-at-scale principle by treating the rare slow path as a first-class result rather than discarding it [1].

A Experiment reporting sheet

This appendix is intentionally a printable reporting aid rather than an evaluation. It makes the evidence required for a future claim visible before a benchmark is run. Copy it into each experiment record together with raw files.

B Measurement decision checklist

1. Is the profiling manifest complete and checked in with raw HDR, CSV, and profiler artifacts?
2. Does the comparison use the same model hash, request corpus, shape, warmup rule, offered load, and latency scope?
3. Are failed runs, error rate, fallback rate, maximum, and cold runs reported separately rather than discarded?
4. Is an optimization claim limited to hardware and software actually named in the manifest?
5. Did numerical tolerances and decision flips pass before performance promotion?
6. Is the result reproducible at least five times and has the fallback path been exercised on unsupported capability or injected impairment?

C Artifact schema

A completed JSON manifest should use the repository schema version, record commit and timestamp, provide CPU/GPU/network and OS/CUDA/driver/compiler details, identify the immutable model bundle and fixed shape, and describe duration, warmup, request rate, Poisson arrival, and queue delay. Its results must include latency percentiles, maximum, throughput, error rate, and fallback rate. The generator ignores null results intentionally. CSV quantiles and raw HDR/NSight files should reside beside the manifest or be referenced by stable repository paths.

Table 8: Run identity and environment sheet. Empty cells are missing evidence, not defaults.

Field	Value	Field	Value
Commit		UTC started	
Model bundle hash		Backend/path	
CPU / RAM		GPU / count	
OS / kernel		Driver / CUDA	
Network		Compiler	

Table 9: Load and outcome sheet. Keep cold and warm populations separate.

Field	Value	Field	Value
Arrival distribution / seed		Offered rate	
Duration / warmup		Queue cap	
Corpus / entity mix		Concurrency cap	
p50 / p95 / p99 / p99.9		Maximum	
Throughput / errors / fallbacks		Raw paths	

References

[1] J. Dean and L. A. Barroso. The Tail at Scale. *CACM*, 56(2), 2013. https://doi.org/10.1145/2408776.2408794

[2] NVIDIA. CUDA Graphs, *CUDA C++ Programming Guide*. https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#cuda-graphs

[3] NVIDIA. Asynchronous Concurrent Execution, *CUDA C++ Programming Guide*. https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#asynchronous-concurrent-execution

[4] NVIDIA. *CUTLASS*. https://github.com/NVIDIA/cutlass

[5] Google. Rust usage, *FlatBuffers documentation*. https://flatbuffers.dev/languages/rust/

[6] Linux man-pages. *shmget(2)*. https://man7.org/linux/man-pages/man2/shmget.2.html

[7] Linux man-pages. *shmat(2)*. https://man7.org/linux/man-pages/man2/shmat.2.html

[8] G. Tene. *HdrHistogram*. https://hdrhistogram.github.io/HdrHistogram/

[9] G.-I. Yu et al. Orca: A Distributed Serving System for Transformer-Based Generative Models. *OSDI*, 2022. https://www.usenix.org/conference/osdi22/presentation/yu

[10] V. J. Reddi et al. MLPerf Inference Benchmark. arXiv:1911.02549, 2019. https://arxiv.org/abs/1911.02549

[11] L. Lamport. Proving the Correctness of Multiprocess Programs. *IEEE TSE*, 1977. https://doi.org/10.1109/TSE.1977.229904

[12] M. M. Michael and M. L. Scott. Simple, Fast, and Practical Non-Blocking and Blocking Concurrent Queue Algorithms. PODC, 1996. https://doi.org/10.1145/248052.248106

[13] X. Huang et al. TabTransformer. arXiv:2012.06678, 2020. https://arxiv.org/abs/2012.06678

[14] P. Micikevicius et al. FP8 Formats for Deep Learning. arXiv:2209.05433, 2022. https://arxiv.org/abs/2209.05433

[15] NVIDIA. Dynamic Batching, *Triton Inference Server User Guide*. https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/user_guide/batcher.html

[16] NVIDIA. *Nsight Systems and Nsight Compute Documentation*. https://docs.nvidia.com/nsight-systems/ https://docs.nvidia.com/nsight-compute/